

HW06 — AI-assisted API Testing

Student ID: 23127184 Course: Kiểm thử phần mềm (Software Testing) SUT: EShop —

<<https://github.com/ttbhanh/eshop-sut>> Repository:

<https://github.com/KieuDuyennn/software__testing/tree/hw6/HW06> Date: _TODO_

1. Scope and API selection

#	Pool	Requirement	Endpoint
1	A	FR-01 Account registration	POST /api/register
2	A	FR-06 Product detail	GET /api/products/:id
3	B	FR-11 Order history (user)	GET /api/orders/my-orders, GET /api/orders/:id
4	C	FR-13 Dashboard	GET /api/admin/orders

Why these four, and how uniqueness within the group was confirmed: _TODO_

Why FR-13's API is GET /api/admin/orders: the dashboard has no endpoint of its own — it derives total revenue (the sum of `total_amount` over orders with `status = 'delivered'`) and the total order count from this payload, so that is the API under test.

2. Test environment

Item	Value
Backend	Node.js + Express + SQLite, eshop/backend/server.js
Base URL	http://localhost:3000
Runner	Postman + Newman <code>newman-reporter-htmlextra</code>
Mandatory header	X-Student-Id: 23127184, injected by a collection-level pre-request script
Test data	Re-seeded on every backend start (<code>database.js</code> drops and recreates all tables)
Rate limiter	200 req / 15 min / IP on /api; bypassed with <code>LOADTEST=1</code> during test runs

Seeded accounts: `admin@eshop.com / Admin123!` (admin) and `test@eshop.com / Test1234!` (user). Seeded data: 3 categories, 5 products, 4 coupons.

3. Method

For each API, the four-step pipeline in Section 6 of the brief:

1. Generate with AI — driven step by step through domain partitions, state

transitions, security (SEC-01..SEC-07) and schema validation, never as a single generic prompt. → `docs/phases/*/01-generate.md`

2. Audit — every generated case labelled VALID / INVALID / INCOMPLETE with

reasoning; the invalid and incomplete ones corrected. → `docs/phases/*/02-audit.md`

3. Extend — at least five cases per API that the AI missed, with a

diagnosis of why. → docs/phases/*/03-extend.md

4. Execute — Newman runs, HTML reports, failure triage.

→ docs/phases/*/04-execute.md

4. API 1 — FR-01 Account registration

4.1 Generation

4.2 Audit

4.3 Extension

4.4 Execution

4.5 Bugs found

5. API 2 — FR-06 Product detail

5.1 Generation

5.2 Audit

5.3 Extension

5.4 Execution

5.5 Bugs found

6. API 3 — FR-11 Order history

6.1 Generation

6.2 Audit

6.3 Extension

6.4 Execution

6.5 Bugs found

7. API 4 — FR-13 Admin dashboard

7.1 Generation

7.2 Audit

7.3 Extension

7.4 Execution

7.5 Bugs found

8. Postman features used

See docs/postman/POSTMAN_FEATURES.md. Summarise the list here.

9. CI/CD integration

See docs/cicd/CI_CD_REPORT.md. Include both required runs — one all-passing, one with a failing test case — with screenshots and links.

10. AI-driven test generator (G9.5 Create)

Design and decisions: docs/design/GENERATOR_DESIGN.md Pseudocode: docs/design/generator_pseudocode.py Diagram (self-drawn): docs/design/diagram/ Agent Skill: .claude/skills/api-test-generator/ Demo video: _TODO_

11. Bug summary

See docs/bugs/BUG_REPORT.md for the full reports and the GitHub Issue links.

ID	Requirement	Severity	Title	Issue

12. Test summary

Metric	Value
APIs tested	4
Test cases generated with AI	
Test cases added by the student	
Total	
Executed	
Passed	
Failed	
Bugs found	

13. AI Critique

See docs/AI_CRITIQUE.md (200–300 words) — reproduce it here in full.

14. Appendix — AI Audit Report

See docs/ai-audit/AI_AUDIT.md — attach as an appendix.

15. Appendix — Git commit log

See evidence/git-commit-log.txt.